Article Type: Description (see below for more detail)

The Reproducibility Crisis of LLM Benchmarks in Software Engineering

Gustavo A. Oliva, *Queen's University, Kingston, ON, K7L 3N6, Canada*
Dayi Lin, *Queen's University, Kingston, ON, K7L 3N6, Canada*
Ahmed E. Hassan, *Queen's University, Kingston, ON, K7L 3N6, Canada*

Abstract—The integration of Foundation Models (FMs) such as Large Language Models (LLMs) into Software Engineering (SE) has spurred benchmarks like SWE-Bench, Aider Polyglot, and Terminal-Bench. Yet current reporting practices fail, producing a reproducibility crisis with two fundamental causes: benchmark parameter sensitivity and run-to-run variance in benchmark results. Configuration parameters across model, inference engine, scaffold, and execution environment are routinely under-reported; the cost is direct — our industrial replication of FeatBench raised gold-patch resolution from 37% to 99.8% only after rebuilding unreported harness and environment parameters. Even with full configuration specification, three compounding sources of variance remain (FM inference non-determinism, benchmark harness flakiness, and infrastructure noise), producing shifts large enough to reorder leaderboards. We present a two-tiered solution: immediate recommendations practitioners can adopt today (configuration reporting, distribution reporting, and Oracle/no-op validation), and long-term recommendations requiring community coordination (crowdsourced statistical benchmarking, standardized benchmark engineering).

The rapid advancement of Large Language Models (LLMs) has created a paradigm shift in software engineering (SE),[1], [2] with models that are now capable of understanding real-world large scale codebases and submitting pull-requrests (PRs) with bug fixes and feature implementations. To measure the progres of these models, the community has developed several SE benchmarks, such as Aider Polyglot and SWE-Bench.[3] However, two fundamental problems undermine the reliability of current benchmark reporting practices.

The first problem is **benchmark parameter sensitivity**. Benchmark results are highly sensitive to configuration parameters, many of which are poorly documented. Parameters span a surprising amount of layers: model weights, inference engine, model inference settings, scaffold behavior, and execution environment. TODO: Add the SWEBench OpenHands tool calling thing.

Digital Object Identifier 10.1109/XXX.0000.0000000

The same dynamic appears at the resource layer: Anthropic reports a six-percentage-point swing on Terminal-Bench 2.0 between strict and uncapped memory configurations simply by varying the eval-time resource envelope.**?** Without comprehensive reporting and enforcement of all parameters, published results become almost irreproducible and straight comparisons can be misleading.

The second problem is **run-to-run variance in benchmark results**. Even when all configuration parameters remain constant and enforced, scores can exhibit significant variation across executions. In our experience, this variance has three compounding sources.

(1) LLM Inference Non-Determinism. A common misconception is that setting the temperature parameter to zero ensures deterministic output. However, this approach fails for two reasons. First, greedy decoding (temperature equals zero) does not guarantee determinism across all LLM implementations and hardware configurations. Second, and more critically, several reasoning models fail or produce degraded

results when temperature is set to zero.[4] This makes temperature-based determinism impractical for comprehensive benchmarking.

(2) Benchmark Code and Harness Flakiness. Tests may produce non-deterministic outputs, or test runners may orchestrate tests non-deterministically. For instance, parallel pytest execution via `xdist` can change outcomes depending on scheduling. In our FeatBench experience,[?] running the released harness 20 times with the gold patch produced consistent results for only 42.3% of tasks, with the rest passing or failing arbitrarily across runs.

(3) Infrastructure Noise. Even with identical model, code, and harness, environmental fluctuations shift outcomes: transient memory spikes can hit OOM kill thresholds, network calls can hit latency spikes or rate limits, container startup races complete in different orders, and cluster contention varies with time of day. These fluctuations are often times invisible from outside the run but can flip a task's verdict.

To demonstrate the magnitude of run-to-run variance, we executed the Aider Polyglot benchmark ten times on the same model using identical configurations (see Figure 1). The pass rates varied substantially across runs, with differences large enough to change the ranking of models on Aider's Leaderboard (see Table **??**). In the current era of "benchmaxing," where every decimal point is scrutinized and minor improvements are celebrated, this level of variation is deeply problematic. The implication is clear: single-point benchmark results are fundamentally unreliable. A model that scores 65% on one run might score 62% or 68% on another. Since the observed spread compounds the three sources above, single-point reporting is even less defensible than the temperature-zero discussion alone implies.

Together, these two problems lead to a **benchmark reproducibility crisis**. Without knowing performance distributions, we cannot meaningfully compare models or track progress. Without comprehensive configuration reporting, we cannot reproduce published results or understand whether improvements stem from genuine model capability advances or undisclosed parameter tuning. Current benchmarking practices provide neither the reproducibility nor the statistical rigor necessary for scientific progress or practical decision-making.

This crisis affects multiple stakeholders across the LLM ecosystem. Model vendors (e.g., OpenAI, Anthropic, DeepSeek) cannot easily benchmark their models against competitors' published results, forcing them to conduct expensive inhouse evaluations. Practitioners evaluating models for production deployment lack the transparency needed to make informed decisions and validate vendor claims. Academic researchers and teams fine-tuning models cannot establish trustworthy baselines to measure their improvements.

In this paper, we present a two-tiered solution framework addressing the two aforementioned fundamental problems. We offer immediate recommendations that individual researchers and companies can adopt today: comprehensive configuration reporting captured in machine-readable files, and mandatory performance distribution reporting (mean, median, standard deviation) instead of single values. We also propose long-term recommendations requiring community coordination: crowdsourced statistical benchmarking infrastructure that exposes performance distributions across multiple submissions, and standardized benchmark engineering that treats benchmarks as robust, interoperable software tools. Our goal is to move the community towards a more robust, reliable, and scientific foundation for evaluating LLMs in software engineering.

RECOMMENDATIONS

We organize our recommendations into two tiers based on their implementation timeline and coordination requirements. Immediate recommendations can be adopted today by individual researchers and companies. Long-term recommendations require community-wide coordination but offer transformative improvements to the benchmarking ecosystem.

IMMEDIATE RECOMMENDATIONS

These recommendations address the most pressing issues and can be implemented immediately without requiring community-wide coordination.

Comprehensive and Standardized Configuration Reporting

A fundamental barrier to reproducibility is the ad-hoc and incomplete manner in which benchmark configurations are currently reported. To address this, we propose a standardized, multi-part reporting structure that covers all layers of the evaluation stack. This structure should be captured in a `benchmark_spec.yaml` file to ensure clarity and consistency.

Model Specification Trust in benchmark results begins with knowing precisely which model was evaluated. Currently, submissions often only specify a

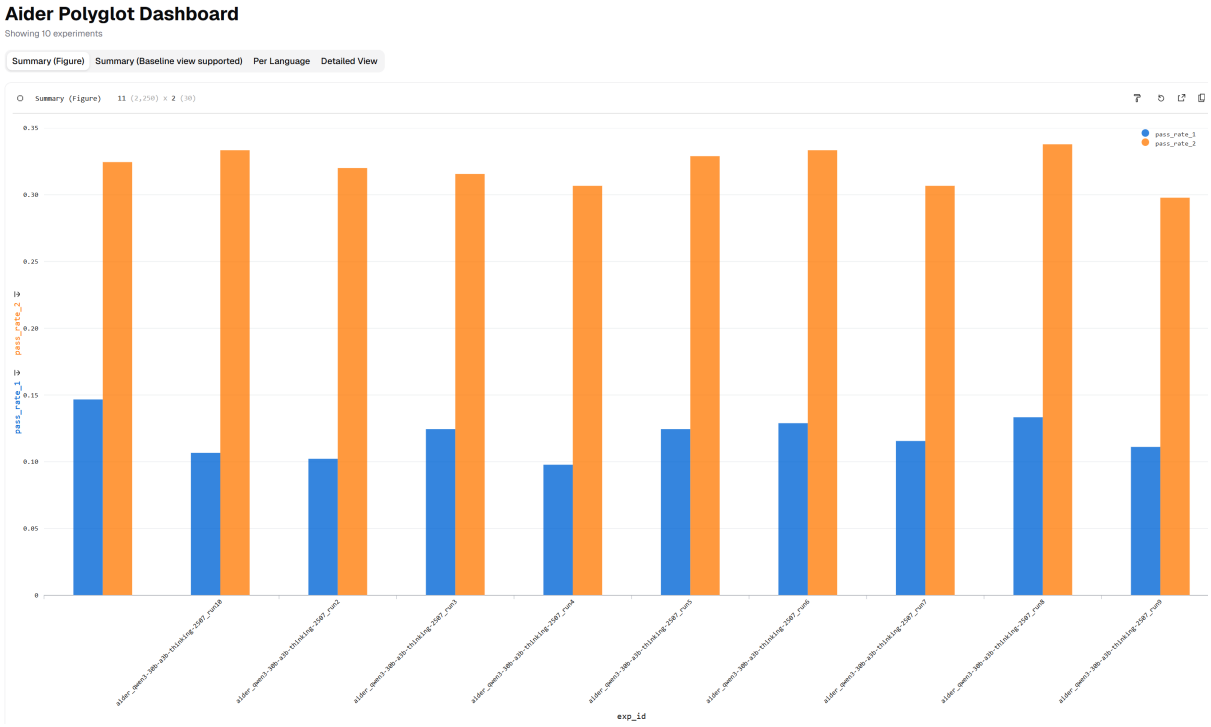

FIGURE 1. Pass rates across ten independent runs of the Aider Polyglot benchmark using identical configurations. The variation in results demonstrates inherent non-determinism in LLM evaluation, with differences large enough to affect model rankings on public leaderboards.

model's name (e.g., "Llama-3-70B"). This is insufficient, as fine-tuned variants can have vastly different capabilities. We propose that all submissions must specify the exact base model and include a cryptographic hash (e.g., SHA-256) of the model weights for any open-source model. This provides an unambiguous, verifiable fingerprint of the exact artifact used.

Inference Engine Parameters The software used to serve the model can significantly impact performance. Parameters related to the deployment and optimization of the inference engine, such as the configuration of vLLM or TensorRT-LLM, are often overlooked. This includes settings like the level of quantization, data-parallel size, and other engine-specific tunings. These parameters must be explicitly reported to understand the full context of the execution environment.

Inference Parameters The parameters used at the time of generation directly control the model's output behavior. While some benchmarks report basic settings like temperature, a complete picture is necessary. We recommend the mandatory reporting of all inference parameters, including but not limited to temperature, top-p, top-k, repetition penalty, maximum token count, and the number of candidate solutions sampled per problem (which feeds directly into how the score metric is computed, as discussed below).

Scaffold Parameters The "scaffold" or "harness" that orchestrates the interaction between the model and the benchmark tasks is a major source of variability. For instance, agentic scaffolds like OpenHands for SWE-Bench have their own configuration parameters that can dramatically influence outcomes. These settings, which control aspects like the agent's strategy or the number of interaction turns, must be treated as first-class citizens in the configuration report.

Execution Environment The execution environment is the most under-reported and highest-leverage layer of the configuration space, yet it can shift scores by large margins. We recommend documenting two dimensions:

– Sandbox and Hardware. The exact sandbox image (e.g., a Docker image hash), the host operating system and kernel version, the hardware family, and the runtime (Docker, containerd, etc.) must be reported.

For non-containerized runs, this includes the Python version, the version of critical drivers such as CUDA, and the full output of `pip freeze` or equivalent dependency lock.

– Resource Envelope. Memory and CPU enforcement deserves explicit attention. Recent industrial experience[?] shows that the gap between a guaranteed allocation (floor) and a hard-kill threshold (ceiling) materially shifts results, with monotonic improvement as ceilings rise. We recommend reporting both values, along with CPU and concurrency caps, per-task time limits, and any kill-on-OOM policy.

Score Metrics A benchmark score is treated as an objective number, yet both its definition and its calculation embed choices that materially shift results. We recommend that score metrics be reported with the same rigor as the model and the execution configuration.

– Definition and Calculation. A metric's name is not its definition, and the calculation behind a familiar-sounding number often embeds choices that move scores independently of the model. The calculation grows trickier when the benchmark setup samples multiple candidate solutions per problem and aggregates them into a single score: the aggregation rule itself becomes a metric decision. Some benchmarks adopt non-standard variants of familiar metrics for this purpose. LiveCodeBench,[?] for example, reports "pass@1" using an estimator over n candidate solutions per problem, rather than evaluating only a single submitted solution. More generally, code-generation pass@k is often defined as the probability that at least one of k sampled candidates passes, estimated from all generated samples for a problem and then averaged across problems. This differs from the common information-retrieval interpretation of pass@k or hit@k, which usually asks whether a relevant item appears in the top k ranked results. Thus, even a familiar name such as pass@k may not be self-explanatory without the exact definition and aggregation procedure. Other metrics carry input parameters that look innocuous but materially shift scores. Our analysis of Codeforces-based Elo evaluations[?] found that the same model on the same contest can shift by up to 394 Elo points purely based on the order in which its failed and accepted submissions are presented to the rating procedure. The model's outputs are unchanged: the swing originates entirely in the metric pipeline.

A separate class of complexity arises when the score calculation depends on external systems. Metrics that delegate correctness to a third-party API or oracle must be documented alongside version pins for that dependency. AI-as-a-judge metrics deserve particular care because they range from a single prompt sent to one model (relatively easy to describe) to multi-stage workflows involving more than one model and custom adjudication logic (effectively their own software systems). We recommend that AI-as-a-judge components be properly packaged so that they can be reused, rather than embedded as ad-hoc scripts. The standardized configuration should then include a reference to the judge implementation (for example, a GitHub repository URL pinned to a commit) along with its own input-parameter choices, such as which models are used in each stage.

– Task Population. Separately from how a score is computed, publications must specify the population it is computed over. The full benchmark task set is a starting point, not a default. Any filtering applied to that set must be reported, whether exclusionary (dropping tasks known to be broken or to have unreliable oracles) or inclusionary (restricting to a subset defined by difficulty, language, repository, or other metadata). Within the chosen population, the score's denominator must then be broken down into three categories: *valid tasks* where the model produced a verifiable result, *constraint violations* where execution exceeded a documented memory, CPU, or time limit constraint, and *infrastructure errors* where the task could not be evaluated due to unexpected environment failures (unexpected OOM events, API timeouts, sandbox setup errors such as failed dependency installation, and network failures). Note that some infrastructure errors may even strike before the model has a chance to generate output. Silently folding them into the denominator deflates the score, while silently dropping them inflates it, and either choice hides a real failure mode of the benchmark itself.

Report Score Distributions, Not Single Values

As demonstrated in our motivating example, inherent non-determinism means that single-point benchmark results are fundamentally unreliable. We recommend that all benchmark publications, whether in research papers or on model leaderboards, must report results from multiple runs rather than a single execution. Specifically, we propose the following requirements.

Minimum Run Count

Benchmark results should be reported based on a minimum of three to five independent runs using identical configurations. While this increases computational

cost, it is the only way to understand the true performance distribution and account for stochastic variation.

Statistical Measures
Instead of reporting a single number, publications must include the mean, median, and standard deviation of the pass rate (or other primary metric) across all runs. This provides a complete picture of both central tendency and variability. For instance, a model with a mean pass rate of 65% and a standard deviation of 3% is fundamentally different from one with the same mean but a standard deviation of 8%.

Critique of Single-Number Leaderboards
Public leaderboards that display only a single score per model are misleading and should be considered unreliable. Without understanding the variance, we cannot know whether a one-point difference between two models is meaningful or simply noise. Leaderboard maintainers should require submitters to provide multiple run data and display confidence intervals or ranges alongside point estimates.

This recommendation serves as a practical mitigation strategy that can be implemented immediately while the community works towards the long-term solution of crowdsourced statistical benchmarking. Even in the absence of a centralized repository, individual researchers and companies can adopt this practice to provide more trustworthy results.

LONG-TERM RECOMMENDATIONS

While the immediate recommendations can be adopted today, the following recommendations require community-wide coordination and infrastructure development. However, they offer transformative improvements to the benchmarking ecosystem.

Shift to Crowdsourced, Statistical Benchmarking

The current model of relying on a few single-point results published in papers or by model vendors is fragile and fails to address both problems identified in our motivating example. We advocate for a paradigm shift towards a crowdsourced, statistical model inspired by mature fields like hardware benchmarking. We envision a central, public repository where the community can submit benchmark results, each accompanied by its detailed configuration file (as per our immediate recommendation on configuration reporting).

This approach directly addresses inherent nondeterminism by making variance visible and quantifiable. Instead of asking "what score did Model X achieve?", we would ask "what is Model X's performance distribution?" With sufficient community submissions, we could establish robust statistical measures (median, interquartile range, outlier detection) that provide a much more reliable signal of true model capabilities.

Simultaneously, this system addresses parameter sensitivity by exposing the impact of different configurations as well as identifying which specific parameters significantly affect results. When the repository contains multiple submissions for the same model with different parameter sets, we can empirically observe how sensitive performance is to specific settings and discover the high-impact parameters that require careful reporting. This transparency would discourage cherry-picked "hero runs" and reveal whether reported improvements stem from genuine model capabilities or merely undisclosed parameter tuning.

Such a system would serve as a powerful reality check for industrial teams. When evaluating whether to adopt a new model, we could compare our internal benchmark results against the community distribution to understand whether our outcomes are typical or anomalous. This would dramatically reduce wasted effort chasing reproducibility issues.

Standardized Benchmark Engineering

A benchmark is a measurement instrument. Like any instrument, it should be calibrated, standardized, and physically separable from the things it measures. Current SE benchmarks are usually none of these. Each is a standalone, idiosyncratic system with its own setup, parameters, and harness, producing wasted effort across organizations and obscuring whether a score reflects the model or the artifact. The two recommendations below address how the engineering of benchmarks must change to make our parameter-reporting and distribution-reporting recommendations operationally feasible at scale. A complementary set of benchmark-engineering practices, broader than but related to these recommendations, is summarized in the sidebar.

Standardize the Interface
A benchmark's interface has two faces: what it declares and what it enforces. On the declarative side, benchmarks should expose a common spec format (e.g., `benchmark_spec.yaml`) capturing all configuration parameters mechanically and reproducibly. On the enforcement side, the harness should validate at startup that the actual run matches the declared spec (image hash, resource caps, kernel version, sandbox enforcement mode), and refuse to proceed if they

Sidebar: Benchmark Hygiene Beyond Reproducibility

Reproducibility is necessary but not sufficient for trustworthy benchmark results. Two further engineering practices apply regardless of how scores are reported.

Validate the benchmark. Before any model score is interpretable, the benchmark itself must be shown to work. We recommend two zero-cost validation runs alongside every published score: an Oracle run submitting the human-authored gold patch (should resolve close to 100% of tasks) and a no-op run submitting an empty patch (should resolve 0%). Our FeatBench experience[?] measured an Oracle pass-rate of 37% on the released harness, rising to 99.8% only after sustained engineering work. The gap between an observed Oracle rate and 100% is the benchmark's noise floor and bounds every downstream claim.

Strengthen the tests. A reliable harness still produces noisy scores if its tests do not distinguish correct solutions from incorrect ones. A recent audit of code-editing benchmarks[?] found median test counts as low as four per problem, with 42.9% of problems below 75% diff-region coverage and 73% of universally-unsolved problems traceable to benchmark artifacts rather than model limits. Benchmark authors should report per-problem test coverage and verify behavior preservation (fail-before, pass-after), not just edit insertion.

diverge. Environment parameters are particularly easy to mis-honor because they are set externally by the runtime rather than by the benchmark code itself, making automated cross-checks especially valuable there. Containerization helps but is not by itself sufficient: different runtimes and host kernels produce different results inside otherwise identical images. The standardization that matters is at the interface level (inputs, outputs, harness contract), not just the packaging.

Decouple Tasks from Harness and Infrastructure
A benchmark task, the harness that runs it, and the execution backend that hosts it should be independently replaceable. Today they are usually fused, so a harness fix forks the dataset and an infrastructure change rewrites the harness. Initiatives such as Harbor[?] are moving in the right direction: a single set of modular interfaces for environments, agents, and tasks, with pluggable backends (local Docker, Daytona, Modal, E2B, Runloop, Tensorlake) so the same task definition runs on the same agent across infrastructures with consistent semantics. The point is not to endorse any single project but to argue that the architectural pattern (task, harness, and infrastructure as separable, swappable concerns) is what reproducible benchmarking requires.

CONCLUSION

The software engineering community has built remarkable benchmarks that have fundamentally shaped our understanding of LLMs in software engineering. Now we must fix how we use them. Three insights should guide us forward:

– Single-point benchmark results are unreliable. The inherent non-determinism of LLM inference means that reported scores need statistical context. While multiple runs are computationally expensive, model providers publishing official performance results must show distributions (e.g., boxplots) rather than single numbers.

– Configuration transparency is non-negotiable. Every benchmark result must include a complete, machine-readable specification of all parameters across the evaluation stack. Without this, reproducibility remains impossible.

– We need crowdsourced statistical benchmarking infrastructure. The current model of isolated, vendor-reported scores limits our ability to establish trustworthy baselines. Community-driven repositories that expose performance distributions and configuration sensitivities would transform how we evaluate and compare models.

The immediate recommendations can be adopted today by individual practitioners and model providers. The long-term recommendations require coordination, but the path is clear. The transformative potential of LLMs in software engineering deserves an evaluation foundation built on transparency, statistical rigor, and community collaboration.

SUPPLEMENTARY MATERIAL

To practice what we preach, we provide complete configuration specifications for all benchmark executions presented in our motivating examples. These include comprehensive `benchmark_spec.yaml` files

covering model specifications, inference engine parameters, inference parameters, scaffold configurations, and execution environment details. Additionally, we provide a JSON schema defining the structure and required fields for benchmark configuration reporting. We emphasize that this schema is not final but rather a starting point for the community to improve upon through feedback and contributions. All materials are available in our GitHub repository: https://github.com/placeholder/llm-benchmark-reproducibility.

ACKNOWLEDGMENTS

The author(s) would like to thank A, B, and C. This work was supported by XYZ under Grant ###.

REFERENCES

1. A. E. Hassan, G. A. Oliva, D. Lin, B. Chen, Z. Ming, and Jiang, "Towards ai-native software engineering (se 3.0): A vision and a challenge roadmap," 2026. [Online]. Available: https://arxiv.org/abs/2410.06107
2. A. E. Hassan, H. Li, D. Lin, B. Adams, T.-H. Chen, Y. Kashiwa, and D. Qiu, "Agentic software engineering: Foundational pillars and a research roadmap," 2025. [Online]. Available: https://arxiv.org/abs/2509.06216
3. C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models solve real-world software engineering problems?" in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=vfr2p51s4i
4. C. Pipis, S. Garg, V. Kontonis, V. Shrivastava, A. Krishnamurthy, and D. Papailiopoulos, "Wait, wait, wait... why do reasoning models loop?" 2025. [Online]. Available: https://arxiv.org/abs/2512.12895

Example is a researcher at the ABC Corporation, Böblingen, Germany. Her current research interests include a, b, and c. Author received her Ph.D. degree in physics from University. She is a Fellow of the IEEE Computer Society. Contact her at sbauthor@abc.com.

Gustavo A. Oliva is ...

Dayi Lin is

Ahmed E. Hassan is